The Grounding System: one shared concept system for all our apps

What it is

A shared set of concepts, copied from the Cardograph video's method, not its tech stack. Nothing here changes any app's stack. No LangSmith, no Clerk, no Supabase, unless an app already uses them. Each app keeps its own auth, database, and providers. What travels between apps is the contract: the same four components, the same rules, the same verification discipline, implemented small in each repo.

Name: the Grounding System. Promise: every AI output is either proven against something real, or labeled as unknown. Nothing inferred is ever presented as proven.

Non-negotiables (Irfan, 2026-10-29)

- Copy concepts, never the stack. Existing auth, database, and providers stay in every app.
- No LangSmith. Tracing and evals are implemented in our own tables and scripts.
- No new vendor is adopted by any plan without a separate explicit go.
- No production data is touched by building this system. Demo, staging, and throwaway accounts only, until a phase explicitly says otherwise with its own approval.
- Every phase ends with a Judge verdict and a live-browser check on the real surface. Test suites and agent reports are claims, not proof. This is the standing definition of done for all projects.

Component G1: Dependency graph and blast radius

What: a small script per repo that parses the repo's TypeScript with ts-morph (the TypeScript compiler API), builds a directed graph of files and import edges, and answers two queries: imported-by (blast radius: what breaks if this file changes) and imports (dependency chain: what this file relies on). Both walks are breadth-first through the graph, two levels deep by default, because further levels return half the repo.

Rules, taken from the video and confirmed by current practice:

- An edge exists only when the parser resolves an import. File names, folder conventions, and plausible guesses never create edges.
- Every import is classified: resolved inside the repo, skipped, or failed, with a reason kept for failures. Coverage (resolved vs. skipped vs. failed) is reported next to every result, always. A graph missing half its edges still looks fine, so completeness is shown, never hidden.

- The parser imports no web framework, UI library, or database. It takes a directory path and returns portable JSON. It runs from a plain script, not from the app.
- Edge kinds: plain imports, re-exports, dynamic imports with a literal string, and require with a literal path. Non-literal require calls are reported unresolved with a reason.
- Regression rule: before-and-after parser runs on the same repo must produce identical edge lists unless the code actually changed. A parser change that silently changes edges is a defect.
- The graph is also the test selector: given changed files, the reverse walk finds every test that transitively imports them. That is how affected-area testing stops being guesswork.

Shared contract (every repo emits the same shape):

- files: path, kind, line count, folder.
- edges: from, to, kind (import, re-export, dynamic, require), resolved true or false, reason when unresolved.
- coverage: files found, files parsed, files skipped with reasons, imports resolved, imports failed with reasons.
- queries: blast-radius(file, depth 2), dependency-chain(file, depth 2), affected-tests(changed files).

Component G2: Reference verification (the four gates)

What: a deterministic checker that runs over AI output before it reaches a user or a Judge. It needs no model and costs no tokens. It is the video's invented-paths evaluator, generalized into four gates (a structure also used by current grounded-generation research):

1. Evidence existence: every cited identifier (file path, task id, record id, route) must exist in the exact set the model was shown. Anything outside the set is invented. This is set membership, nothing more.
2. Entity allowlisting: every cited entity name (medicine, test, topic, company, person) must come from the app's own allowed list for that request. Paraphrased names from training data fail.
3. Number binding: every cited number (hours, amounts, counts, dates) must exactly match a canonical value surfaced from the app's own data. Rounded, sign-flipped, or reformatted numbers fail.
4. Claim-type rules: a claim of a given type must cite evidence of the required type. A comparison claim must cite the two things compared. A recommendation must cite the evidence it rests on.

Behavior on failure: the output is flagged, not shipped. The checker returns the failed gate, the offending citation, and the allowed set it was checked against, so a human or a repair pass can fix it precisely. The checker never rewrites output silently. Repair is a separate, visible step: generate, check, verify, repair.

What it is not: a judge model. Judge scores (specificity, helpfulness, tone) are signal, not proof, and are always reported separately from the four deterministic gates. A high judge score never overrides a failed gate.

Where it applies per app (each project plan names its own sets): file paths and task ids in DayMirror; medicines, tests, and patient record ids in Clinic; component and screen names in Centher; topics and question ids in Elite JS and Viva.

Component G3: Model-call gateway (trace and content cache)

What: one server module per app through which every model call passes. No call site talks to a provider directly.

- Tracing: every call records input, output, model, provider, token counts, latency, and cost, in our own table. Traces are how cost becomes visible and how prompt changes become comparable.
- Content-keyed caching: the cache key is a SHA-256 hash of everything that determines the answer (the inputs, the full system prompt, the provider, the model, and a schema version). A hit returns the cached answer with zero tokens and zero cost. Invalidation is emergent: change any input and the key changes, so the old entry is never looked up again. Nobody writes "clear cache" logic.
- Cache hits still emit a trace showing a cache hit, so caching is verified, not assumed. Every cached response carries metadata (hit or miss) so cost reports are honest.
- Version 1 is exact-hash caching only. Semantic caching (reusing answers for reworded questions) is explicitly deferred: it needs embeddings, a similarity threshold, and tuning, and a wrong threshold serves wrong answers confidently. Revisit only with measured demand.
- Prompt versions are stored with their schema version, so a current prompt and a retired prompt can be scored side by side by the G2 gates and by judge scores, separately.

Component G4: Spec discipline

What: the working rules for every agent crew, adopted from the video and already close to our standing practice. Written here once so all six project plans inherit them instead of reinventing them.

- Every feature starts as a short spec stating what it does, what it must not do, and how success will be checked. The spec lives in the repo or goal docs before building starts.
- If a build fails its acceptance check, discard the code, fix the spec, and rerun. Do not prompt on top of bad code. A failed acceptance means the spec was wrong.
- When ambiguous, the agent asks one specific question and brings a proposed answer (option A or B). Open-ended questions stall loops. Guesses create silent errors. A specific question with a proposal keeps the decision small and checkable.
- The rules file for agents is kept short, because it is read on every prompt, and it avoids naming folders or structures the agent could then invent.
- Scope discipline: every spec names what is out of scope. For this system, out of scope everywhere in version 1: semantic caching, a visual graph canvas (the script comes first), quality judgments about code (the graph answers structure questions, not quality questions), and any stack migration.

Component G5: Tenant isolation (defense in depth)

What: the multi-tenancy rule for every app that serves more than one team, clinic, company, or user group. This matters most for Clinic, and applies wherever tenant data exists.

- Every tenant-scoped row carries a tenant id (organization, clinic, or company id). Every query is scoped to it.
- The decision belongs in database policy first, not only in application filters. Application filters fail silently when a developer forgets one. Policies fail closed. Where the stack supports row-level security, enable it on every tenant table and write policies that tie rows to the caller's tenant for select, insert, update, and delete, with role checks (owner, admin, member) written as a conjunction inside one policy, because separate permissive policies combine with OR.
- Our stacks do not use Supabase auth, so the Supabase JWT-claim pattern is adapted, not copied: the server derives the tenant from its own validated session (never from request payload), and sets it as a transaction-local setting for the request, so policies read it from the database session. The exact mechanism per app is designed in that app's plan against its real auth (Better Auth or otherwise) and verified by test, not assumed from the video.
- Service keys and admin paths bypass policy by design and must never reach a client. Migrations and bulk work use a separate path with its own audit trail.
- Acceptance test, copied from the video: two tenants, no cross-tenant rows, proven by switching tenants and reading. If another tenant's row ever comes back, the bug is the policy, not the screen. This test is mandatory for Clinic before any multi-clinic data ships, and recommended everywhere else tenant data exists.
- Cross-tenant rules for AI: a model call for one tenant must never receive another tenant's data as context, and G2's allowed sets are built per tenant, per request. Scoped credentials (short-lived, naming one tenant and one object) are used wherever an agent or background job acts across a tenant boundary.

How the 12-agent crew runs this system in each project

Every project plan uses the same crew, already decided: 7 core (Product Manager, Product Designer, Frontend Engineer, Backend Engineer, QA Engineer, Tech Lead, DevOps/Release) plus Engineering Manager (manager gate) plus CEO agent (Irfan's stand-in, gates pricing, positioning, public surfaces, and money flows only; new decisions escalate to Irfan) plus 3 client agents (adapted per product: small-firm admin, mid-market ops manager, frontline employee or equivalent). The captain (Jarvis) coordinates and is not one of the 12.

Gates per phase, in every plan:

- PM writes the spec. Designer reviews every screen and modal through the UI/UX skills with local-ui-ux-director sign-off before it is cleared.
- Backend builds G1/G2/G3 pieces behind the spec. Frontend builds only what the spec names.
- QA runs Guardian tests including real-database concurrency and security probes where the phase touches data or tenants.

- Judge gives APPROVE or the phase does not merge. Numbers must be reproduced, not reported.
- Pilot passes on a live browser on the real user-facing surface, recorded, and analyzed under the post-recording re-analyze loop. A video or demo is not final until a separate analysis pass comes back clean.
- Tech Lead and Engineering Manager gates exist to push back, not just execute. Client agents answer: would this solve your day, would you pay?
- LEARNING.md updated per phase, with a learning note and PDF per definition of done. Test data cleaned up with ids recorded.

Rollout order across projects

1. DayMirror first: the proving ground. The shared core (G1 script, G2 checker, G3 gateway) is built and proven here, where the crew and gates are most mature.
2. Clinic second: the highest-stakes rollout, with the full roadmap in its own plan. G5 tenant isolation and G2 citation checks for the doctor-helper AI are the priority, ahead of any other component.
3. Company OS / Viyacor, Centher, Elite JS, Viva after: each plan adopts the proven core and adds only its own sets, tenants, and call sites. No project rebuilds the core from scratch. No project waits for another project's phases; once the DayMirror core is proven, rollouts run in parallel in their own chats.

Research notes folded into this system (2026-10-29)

- ts-morph plus forward and reverse graphs is the established way to build blast-radius and affected-test queries for TypeScript. Existing tools in this space (impact-mapper style MCP servers, architecture analyzers) confirm the approach and also confirm that the valuable part is the graph contract, not any one tool.
- Row-level security patterns converge on: tenant id on every row, policies for select/insert/update/delete, tenant plus role in one policy, and the tenant taken from validated session data, never from payload. The transaction-local tenant setting is the portable form for stacks without Supabase auth.
- Content-addressed LLM caching (SHA-256 over all answer-determining inputs) makes invalidation emergent and hits provably identical. Semantic caching is a second layer with its own costs and thresholds, deferred in version 1.
- Grounded-generation research converges on deterministic gates (existence, allowlisting, number binding, claim-type rules) plus a generate, check, verify, repair pipeline, with judge models used only for qualities that have no exact answer.

Files in this set

- 00-the-grounding-system.md (this file): the shared system.
- daymirror-plan.md: DayMirror proving-ground plan.

- clinic-os-roadmap.md: full Clinic roadmap, tenant isolation first.
- company-os-viyacor-plan.md, centher-plan.md, elite-js-plan.md, viva-plan.md: rollout plans per project.
- Each file is delivered as Markdown and PDF. Implementation starts in each project's own chat, from its plan, after the plans land.